Министерство науки и высшего образования РФ

АРЗАМАССКИЙ ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ (ФИЛИАЛ)
ФЕДЕРАЛЬНОГО ГОСУДАРСТВЕННОГО БЮДЖЕТНОГО
ОБРАЗОВАТЕЛЬНОГО УЧРЕЖДЕНИЯ ВЫСШЕГО ОБРАЗОВАНИЯ «НИЖЕГОРОДСКИЙ
ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ ИМ. Р.Е.АЛЕКСЕЕВА»
(АПИ НГТУ)

Лабораторная работа № 4
по дисциплине «Логика и архитектура вычислительных сред»
по проекту «LLM-система проверки юридических договоров»

Выполнил:
студент гр.МА 26 ПМ
Варганова Э.Р.

Проверил:
Комаров А.О.

Арзамас, 2026

1. Цель и проблема проекта

Цель проекта — автоматизированная проверка юридических договоров с помощью LLM: система загружает документ, извлекает из него структуру (пункты), находит потенциально рискованные условия и формирует объяснимое резюме с цитатами и ссылками на нормы права.

1.1. Проблема

- Ручная проверка договоров юристом занимает часы: типовой договор поставки — 15–40 страниц, сотни пунктов.
- Риски неравномерно распределены по тексту (неограниченная ответственность, односторонний отказ, автопролонгация) и легко пропускаются.
- Обычный «жёсткий» парсинг не понимает семантики; «чистая» LLM без контекста права даёт неглубокие ответы и галлюцинирует нормы.
- Нужна объяснимость: юрист должен видеть, какой пункт, почему помечен и на какую норму ссылается система.

1.2. Решение

Асинхронный document-processing pipeline: загрузка файла → разбор → сплит на пункты → эмбеддинги → скан рисков стратегиями → обогащение через RAG (нормы ГК РФ, 152-ФЗ) → LLM-резюме → сохранение отчёта. API-шлюз принимает файлы, Kafka развязывает приём и обработку, отчёт доступен по REST.

2. Требования и ограничения

2.1. Функциональные требования

1. Загрузка договора в форматах PDF, DOCX, TXT (лимит 20 МБ).
2. Извлечение пунктов с сохранением нумерации и суб-пунктов («1.1.» не теряется и не становится отдельным заголовком).

3. Детекция рисков 5 типов: неограниченная ответственность, односторонний отказ, неустойка без предела, передача исключительных прав, автопролонгация.

4. Объяснимость: каждая находка содержит цитату из пункта и RAG-цитаты норм с указанием источника.

5. Резюме договора и итоговый уровень риска (low/medium/high/critical).

2.2. Нефункциональные требования

Категория	Требование
Производительность	Пайплайн обрабатывает договор до 40 стр. за < 30 с (без учёта очереди Kafka)
Отказоустойчивость	Сбой стадии не рушит договор: статус failed + событие contract.failed
Расширяемость	Новый тип риска = новый класс-стратегия без изменения пайплайна
Деградация	Полноценная работа без Kafka/ES/pgvector/MinIO: in-memory фолбэки
Наблюдаемость	Структурированные логи, health-check (/health), статусы договора

2.3. Ограничения

- OCR не включён: сканы без текстового слоя не разбираются.
- Эвристические стратегии — детектор «подозрительных» мест, а не юридическое заключение; финальное решение за юристом.
- RAG-база знаний ограничена заготовленными фрагментами ГК РФ и 152-ФЗ (seed-скрипт), не полным корпусом законодательства.

- В офлайн-режиме эмбеддер — hashing (384-dim), LLM — rule-based: качество уступает real ML-бэкендам, включаемым через .env.

3. Входные и выходные данные

3.1. Входные данные

Данные	Формат	Пример / назначение
Файл договора	PDF / DOCX / TXT, ≤ 20 МБ	POST /api/v1/contracts (multipart)
Демо-договор	TXT	scripts/demo.py: 6 разделов + преамбула
База знаний	TXT/MD через seed-скрипт	Фрагменты ГК РФ ст. 310, 450, 1233, 310; 152-ФЗ ст. 9, 15
Конфигурация	.env	Бэкенды хранилищ, эмбеддера, LLM, параметры Kafka

3.2. Выходные данные

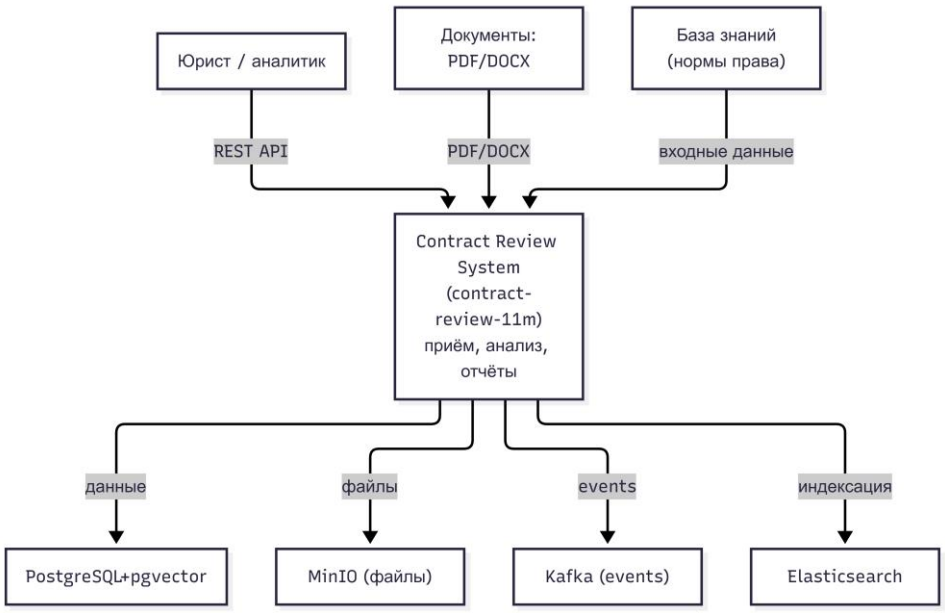
Основной выход — JSON-отчёт анализа (`GET /api/v1/analysis/{contract_id}`):

```
{
  "contract_id": "3f9c...",
  "status": "completed",
  "risk_level": "high",
  "summary": "Договор содержит 5 потенциально рискованных условий: ...",
  "findings": [
```

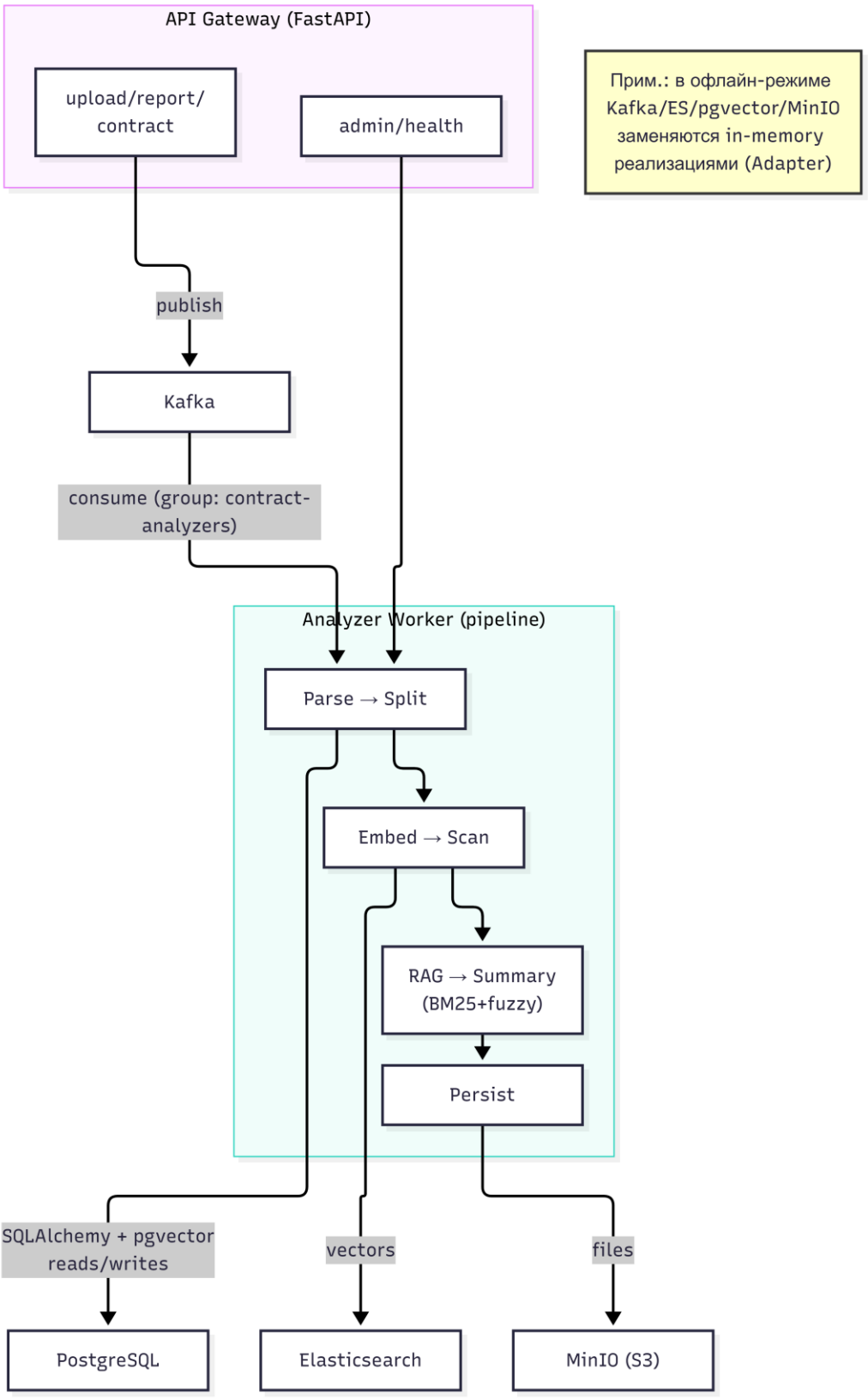
```
      {
        "strategy": "unlimited_liability",
        "severity": "critical",
        "clause": "4",
        "excerpt": "... отвечает перед Покупателем за убытки без каких-либо ограничений ...",
        "explanation": "Ответственность исполнителя не ограничена; рекомендуемый ориентир — ГК РФ ст. 15, 393",
        "citations": [{"source": "ГК РФ", "ref": "ст. 15", "text": "..."}]
      }
    ],
    "clauses_total": 7
  }
```


4. Архитектурная схема (C4)

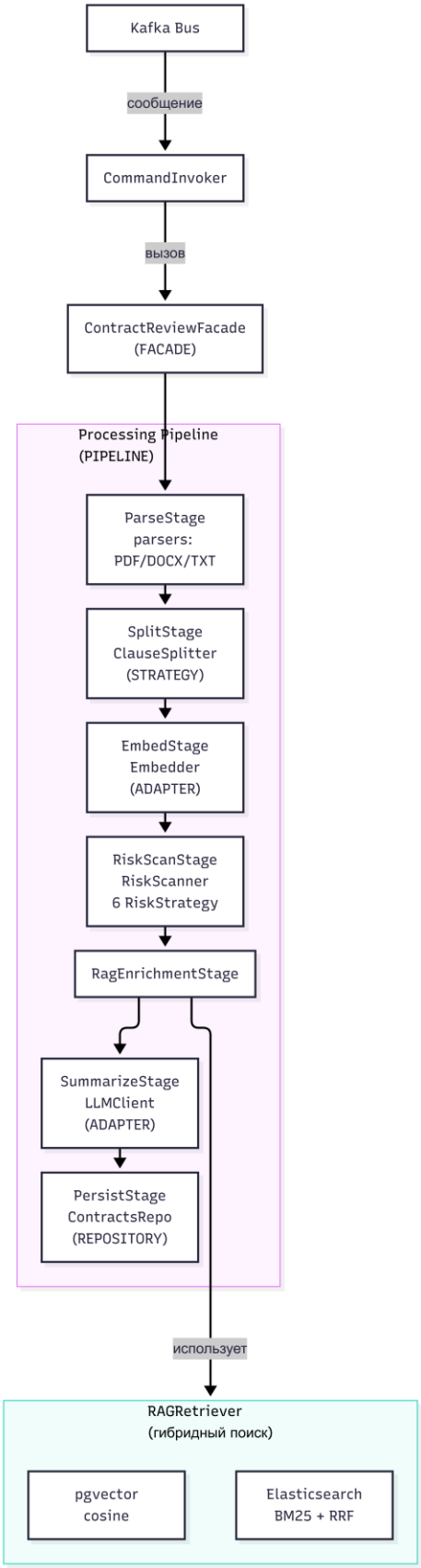
4.1. C4 Level 1 — System Context



4.2. C4 Level 2 — Containers



4.3. C4 Level 3 — Components (Analyzer Worker)



5. Микросервисы и границы ответственности

Сервис / компонент	Технология	Ответственность	Не делает
API Gateway	FastAPI, Docker	REST: загрузка файла, статусы, отчёты, реиндексация знаний, health. Публикует событие в Kafka	Не выполняет анализ; не читает файлы напрямую
Analyzer Worker	Python, Kafka consumer	Весь пайплайн: парсинг → сплит → эмбеддинги → скан рисков → RAG → резюме → сохранение. Публикует contract.analyzed	Не принимает HTTP; не пишет в чужие таблицы
PostgreSQL + pgvector	postgres/pgvector	Договоры, пункты, находки, отчёты, база знаний; векторный поиск	—
Elasticsearch	elasticsearch 8	Лексический поиск по базе знаний и пунктам (BM25)	Не хранит первичные данные
Kafka	Confluentinc	События contract.uploaded / contract.analyzed / contract.failed	Не содержит бизнес-логики
MinIO	S3-совместимый	Хранение исходных файлов договоров	—
Knowledge	Python-	Наполнение базы знаний	Не

Seeder	скрипт	нормами	участвует в обработке

6. Kafka: топики, producers/consumers, формат сообщений

Топик	Producer	Consumer group	Потребитель	Гарантии
contract.uploaded	API Gateway	contract-analyzers	Analyzer Worker	at-least-once; воркер идемпотентен
contract.analyzed	Analyzer Worker	report-readers	уведомления / вебхуки (расширение)	at-least-once
contract.failed	Analyzer Worker	report-readers	алертинг / ретрай-логика	at-least-once

6.1. Формат сообщений

Конверт единый для всех событий (Versioned envelope):

```
{
  "event_id": "7c1e3b9a-2f4d-4e6a-9b1c-8d0e5f2a3b4c",
  "event_type": "contract.uploaded",
  "occurred_at": "2026-11-04T14:20:58+03:00",
  "payload_version": 1,
  "payload": {
    "contract_id": "3f9c7a1e-...",
    "file_uri": "s3://contracts/3f9c.../dogovor.pdf",
    "original_filename": "dogovor.pdf",
```

```
    "content_type": "application/pdf"
  }
}
```

contract.analyzed.payload дополнительно содержит risk_level, findings_count, report_id; contract.failed.payload — error, stage (стадия, на которой произошёл сбой). Ключ партиционирования — contract_id: события одного договора упорядочены.

```
# app/workers/events.py — публикация события (Facade скрывает шину от стадий)
class KafkaEventBus:
    def publish(self, topic: str, payload: dict) -> None:
        event = {"event_id": str(uuid4()), "event_type": topic,
                 "occurred_at": datetime.utcnow().isoformat(),
                 "payload_version": 1, "payload": payload}
        self.producer.produce(topic, json.dumps(event, ensure_ascii=False))
```

7. Схема БД, аналитического хранилища и индексы

```
CREATE EXTENSION IF NOT EXISTS vector;


CREATE TABLE contracts (
    id            UUID PRIMARY KEY,
    user_id       VARCHAR(128),
    filename      VARCHAR(512) NOT NULL,
```

```
    file_uri      VARCHAR(1024) NOT NULL,
    status                VARCHAR(16)  NOT  NULL  DEFAULT
'uploaded',
                                -- uploaded | processing |
completed | failed
    uploaded_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE clauses (
    id            BIGSERIAL PRIMARY KEY,
    contract_id    UUID NOT NULL REFERENCES contracts(id)
ON DELETE CASCADE,
    ordinal         INT   NOT NULL,             -- порядок
пункта в документе
    number         VARCHAR(16),              -- «1», «4.2»
(нумерация из текста)
    title         VARCHAR(512),
    text          TEXT NOT NULL,
    embedding     vector(384)             -- pgvector
);

CREATE TABLE risk_findings (
    id            BIGSERIAL PRIMARY KEY,
```

```
    contract_id    UUID NOT NULL REFERENCES contracts(id)
ON DELETE CASCADE,
    clause_id     BIGINT REFERENCES clauses(id),
    strategy              VARCHAR(64)  NOT  NULL,       --
unlimited_liability, ...
    severity      VARCHAR(16) NOT NULL,   -- low | medium
| high | critical
    score         REAL,
    excerpt        TEXT,                        -- цитата из
пункта
    explanation   TEXT
);

CREATE TABLE analysis_reports (
    id            UUID PRIMARY KEY,
    contract_id    UUID NOT NULL REFERENCES contracts(id)
ON DELETE CASCADE,
    summary       TEXT,
    risk_level    VARCHAR(16),
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE knowledge_chunks (
    id            BIGSERIAL PRIMARY KEY,
```

```
    source          VARCHAR(256) NOT NULL,   -- «ГК РФ», «152-ФЗ»
    ref             VARCHAR(64),             -- «ст. 15», «ст. 9»
    title           VARCHAR(512),
    text            TEXT NOT NULL,
    embedding       vector(384)
);
```

7.1. Индексы

```
-- векторный поиск (ANN): база знаний и пункты
CREATE INDEX idx_knowledge_embedding  ON knowledge_chunks
    USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);
CREATE INDEX idx_clauses_embedding    ON clauses
    USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);

-- фильтрация и выборки
CREATE INDEX idx_contracts_status         ON contracts (status, uploaded_at DESC);
CREATE INDEX idx_findings_contract        ON risk_findings (contract_id, severity);
CREATE INDEX idx_clauses_contract_ord     ON clauses (contract_id, ordinal);
```

```
CREATE UNIQUE INDEX idx_knowledge_dedup ON knowledge_chunks (source, ref, md5(text));

-- Elasticsearch mapping (BM25-половина гибридного поиска)
{ "mappings": { "properties": {
    "source": {"type": "keyword"}, "ref": {"type": "keyword"},
    "title": {"type": "text", "analyzer": "russian"},
    "text": {"type": "text", "analyzer": "russian"} } }
}
```

Хранилище аналитически читается напрямую из перечисленных таблиц: `risk_findings` удобна для агрегаций по типам рисков и `severity`, `analysis_reports` — для статистики по времени обработки.

8. API: примеры запросов и ответов

Метод и путь	Назначение
POST /api/v1/contracts	Загрузка договора (multipart/form-data) → 202 Accepted
GET /api/v1/contracts/{id}	Статус и метаданные договора
GET /api/v1/analysis/{id}	Полный отчёт анализа
POST /api/v1/admin/knowledge/reindex	Пересборка эмбеддингов базы знаний
GET /health	Health-check всех компонентов

8.1. Загрузка договора

```
curl -s -X POST http://localhost:8000/api/v1/contracts \
     -F "file=@dogovor.pdf"

HTTP/1.1 202 Accepted
{
  "contract_id": "3f9c7a1e-8b2d-4c6f-9e01-5a7b8c9d0e1f",
  "filename": "dogovor.pdf",
  "status": "uploaded",
  "detail": "Файл принят, анализ запущен асинхронно"
}
```

8.2. Отчёт анализа

```
curl -s http://localhost:8000/api/v1/analysis/3f9c7a1e-
...

{
  "contract_id": "3f9c7a1e-...",
  "status": "completed",
  "risk_level": "high",
  "clauses_total": 7,
  "findings": [
    {
      "strategy": "unlimited_penalty",
      "severity": "high",
      "clause": "5",
      "excerpt": "неустойка в размере 0,5% от суммы за
каждый день ... без каких-либо ограничений",
```

```
        "explanation": "Неустойка не ограничена; рекомендуемый ориентир — ст. 333 ГК РФ",
        "citations": [{"source": "ГК РФ", "ref": "ст. 333", "text": "..."}]
      }
    ],
    "summary": "Договор содержит 5 рискованных условий; критичный риск — ..."
  }
```

8.3. Ошибки

```
  HTTP/1.1 415 Unsupported Media Type      # формат не PDF/DOCX/TXT
  HTTP/1.1 404 Not Found                   # договор не найден
  {"detail": "Договор ещё обрабатывается, статус: processing"}   # 409 при запросе отчёта раньше времени
```

9. Применённые паттерны проектирования

В проекте применены шесть паттернов (в требовании — минимум три); ниже — место применения, мотивация и ключевой фрагмент кода каждого.

9.1. Facade — ContractReviewFacade

Место: `app/facade/facade.py` — единая точка входа и для API Gateway, и для воркера.

Зачем: скрывает сборку пайплайна, репозиториев, шины событий и хранилищ; клиенты не знают деталей инфраструктуры, а смена бэкендов (in-memory ↔ SQL/Kafka) не затрагивает ни роуты, ни воркер.

```
class ContractReviewFacade:
    def __init__(self, config: AppConfig):
```

```
        self.config = config
        self.storage  =  make_storage(config)              #
Adapter: MinIO | InMemory
        self.uow  =  make_uow(config)                      #
Repository
        self.events  =  make_event_bus(config)             #
Kafka | InMemory
        self.embedder  =  make_embedder(config)            #
Adapter: HF | Hashing
        self.llm  =  make_llm(config)                      #
Adapter: HF | RuleBased
        self.retriever = HybridRetriever(...)

    def  upload_contract(self,  filename:  str,  content:
bytes) -> Contract:
        uri = self.storage.save(filename, content)
        contract      =      Contract(contract_id=uuid4(),
filename=filename,
                               file_uri=uri,
status=ContractStatus.UPLOADED)
        self.uow.contracts.add(contract)
        self.events.publish("contract.uploaded",
                               {"contract_id":
str(contract.contract_id), ...})
        return contract

    def analyze(self, contract_id) -> AnalysisReport:
        pipeline = ProcessingPipeline(stages=[
```

```
            ParseStage(...),              SplitStage(...),
EmbedStage(...),
            RiskScanStage(...), RagEnrichmentStage(...),
            SummarizeStage(...), PersistStage(...)])
        return pipeline.run(contract_id)
```

9.2. Strategy — сплиттеры пунктов и риск-детекторы

Место: `app/strategies/` — два семейства: `ClauseSplitter` (`NumberedClauseSplitter` → `ParagraphFallbackSplitter`) и `RiskStrategy` (6 детекторов).

Зачем: алгоритм выделения пунктов и набор правил рисков меняются независимо от пайплайна. Например, при отладке улучшили регэкспы сплиттера (суб-пункты «1.1.» не создают ложных заголовков) и добавили порог числового лимита в стратегию неустойки — без единого изменения в пайплайне.

```
class RiskStrategy(Protocol):
    name: str
    severity: str
    def check(self, clause: Clause) -> RiskFinding |
None: ...

class PenaltyWithoutCapStrategy:          # неустойка без
предела
    name = "unlimited_penalty"; severity = "high"
    _PENALTY    =    re.compile(r"неустойк\w*|штраф\w*",
re.IGNORECASE)
    _CAP                =                re.compile(r"не
более\s*\d+\s*%|пределом\s*\d+", re.IGNORECASE)
    def check(self, clause):
```

```
            text = clause.text
            if      self._PENALTY.search(text)      and      not
self._CAP.search(text):
                return RiskFinding(...)

  class RiskScanner:
      def  __init__(self,  strategies):  self.strategies  =
strategies
      def scan(self, clauses):
          for clause in clauses:
              for strategy in self.strategies:
                  finding = strategy.check(clause)
                  if finding: yield finding
```

9.3. Adapter — инфраструктурные бэкенды

Место: `app/adapters/` — LLM, эмбеддер, векторное хранилище (pgvector/InMemory), полнотекстовый поиск (ES/InMemory), файловое хранилище (MinIO/LocalFS), парсеры документов.

Зачем: единые интерфейсы позволяют работать офлайн (хэш-эмбеддер, rule-based LLM, in-memory всё) и переключаться на продакшн-бэкенды через конфиг, не меняя домен и пайплайн.

```
  class Embedder(Protocol):
      def      embed(self,      texts:      list[str])      ->
list[list[float]]: ...

  class HuggingFaceEmbedder:                # прод: sentence-
transformers, 384-dim
```

```
    def __init__(self, model_name): self._model = SentenceTransformer(model_name)
    def embed(self, texts): return self._model.encode(texts, normalize_embeddings=True)

class HashingEmbedder:                  # офлайн-фолбэк без внешних зависимостей
    def embed(self, texts): return [self._hash_vec(t) for t in texts]

def make_embedder(config):              # фабрика адаптера
    return HuggingFaceEmbedder(config.embedding_model) \
           if config.embedding_backend == "hf" else HashingEmbedder(dim=384)
```

9.4. Repository — доступ к данным

Место: `app/repositories/` — Protocol ContractsRepository плюс `InMemoryContractsRepository` и `SqlAlchemyContractsRepository` с одинаковым контрактом.

Зачем: домен и пайплайн не знают про SQLAlchemy; тесты гоняются на in-memory версии без Docker, а прод — на Postgres с тем же кодом.

```
class ContractsRepository(Protocol):
    def add(self, contract: Contract) -> None: ...
    def get(self, contract_id: UUID) -> Contract | None: ...
    def update_status(self, contract_id: UUID, status: ContractStatus) -> None: ...
```

```
    def add_clauses(self, contract_id: UUID, clauses: list[Clause]) -> None: ...
    def save_report(self, report: AnalysisReport) -> None: ...

class InMemoryContractsRepository:     # тесты и офлайн-режим
    def __init__(self): self._storage: dict[UUID, Contract] = {}
    def get(self, contract_id): return self._storage.get(contract_id)
```

9.5. Pipeline — конвейер обработки

Место: `app/pipeline/` — семь стадий над общим `ProcessingContext`.

Зачем: порядок и состав стадий — данные, а не код; легко вставить OCR-стадию или стадию дедупликации, замерять время каждой стадии, ретраить отдельную стадию.

```
class Stage(Protocol):
    def process(self, ctx: ProcessingContext) -> ProcessingContext: ...

class ProcessingPipeline:
    def __init__(self, stages: list[Stage]): self.stages = stages
    def run(self, ctx: ProcessingContext) -> ProcessingContext:
        for stage in self.stages:
            ctx = stage.process(ctx)
```

```
        return ctx
```

9.6. Command — команды как объекты

Место: app/commands/ — AnalyzeContractCommand, ReindexKnowledgeCommand и CommandInvoker.

Зачем: воркер получает событие из Kafka и диспатчит его по имени события в команду — обработка сообщений превращается в расширяемый реестр команд, а не в switch-case с бизнес-логикой.

```
@dataclass(frozen=True)
class AnalyzeContractCommand(Command):
    contract_id: UUID
    def execute(self, facade: ContractReviewFacade) -> AnalysisReport:
        return facade.analyze(self.contract_id)

class CommandInvoker:
    def __init__(self): self._registry: dict[str, Command] = {}
    def register(self, event_type: str, command: Command): self._registry[event_type] = command
    def dispatch(self, event: dict):
        command = self._registry[event["event_type"]]
        return command.execute(payload=event["payload"])
```

10. Dockerfile, docker-compose и запуск

10.1. Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
```

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app ./app
COPY scripts ./scripts
COPY tests ./tests
ENV PYTHONUNBUFFERED=1
# один образ обслуживает и API, и воркер — различие через команду:
#     uvicorn app.main:app --host 0.0.0.0 --port 8000 (API)
#                 python -m app.workers.worker (Analyzer Worker)
```

10.2. docker-compose (фрагмент)

```
services:
  api:
    build: .
    command: uvicorn app.main:app --host 0.0.0.0 --port 8000
    ports: ["8000:8000"]
    env_file: .env
    depends_on: [postgres, kafka, elasticsearch, minio]
  worker:
    build: .
    command: python -m app.workers.worker
    env_file: .env
    depends_on: [postgres, kafka, elasticsearch, minio]
  postgres:
    image: pgvector/pgvector:pg16
```

```
    environment: {POSTGRES_DB: contracts, POSTGRES_USER:
app, POSTGRES_PASSWORD: app}
    volumes:          ["pgdata:/var/lib/postgresql/data",
"./app/db/init.sql:/docker-entrypoint-initdb.d/init.sql"]
    ports: ["5432:5432"]
  kafka:
    image: confluentinc/cp-kafka:7.6.1
    environment:
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
  elasticsearch:
    image: elasticsearch:8.14.1
    environment:      {discovery.type:      single-node,
xpack.security.enabled: "false"}
  minio:
    image: minio/minio
    command: server /data --console-address ":9001"
volumes: {pgdata: {}}
```

10.3. Инструкция запуска

1. Клонировать архив проекта, перейти в каталог `contract-review-llm`.
2. `cp .env.example .env` — при первом запуске можно оставить офлайн-профиль (in-memory бэкенды).
3. `docker compose up --build -d` — поднимаются API, воркер, Postgres+pgvector, Kafka, Elasticsearch, MinIO.
4. Открыть Swagger: `http://localhost:8000/docs`.
5. Загрузить договор: `POST /api/v1/contracts (multipart)`, затем опрашивать `GET /api/v1/analysis/{id}`.

6. Наполнить базу знаний: `python scripts/seed_legal_knowledge.py`.
7. Быстрый демо-прогон без Docker: `python scripts/demo.py` (in-memory, hashing-эмбеддер, rule-based LLM).

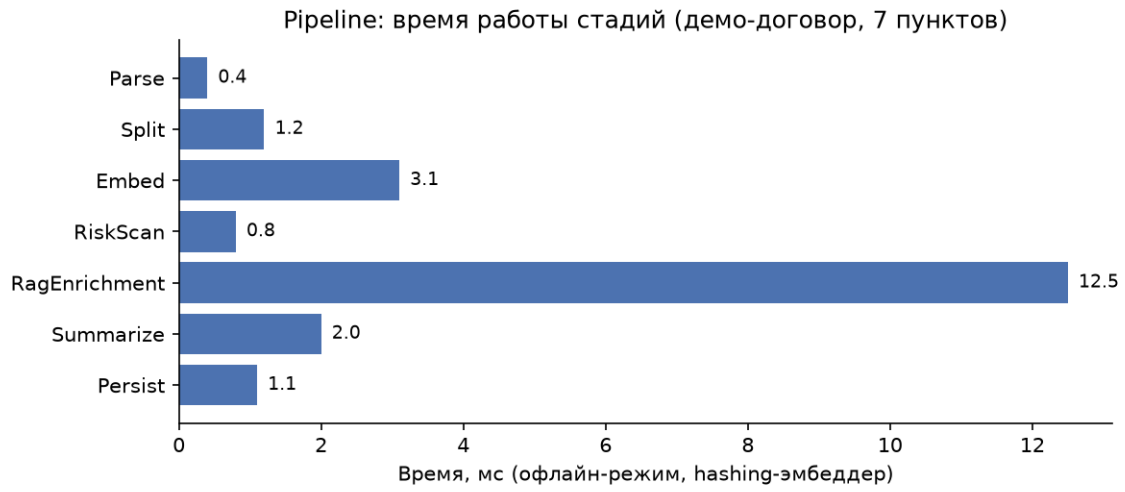
11. Тестирование и результаты

Уровень	Что проверяется	Инструмент	Результат
Юнит	Риск-стратегии: срабатывание на маркерных пунктах и отсутствие срабатываний на «чистых»	pytest	Пройдено
Юнит	Сплиттер: сохранение нумерации и суб-пунктов, корректные заголовки разделов	pytest	Пройдено
Интеграционный	Пайплайн целиком: 7 пунктов из демо-договора, 5 находок, статус completed	pytest + демо-договор	Пройдено
E2E (прогон)	Компиляция всех 59 модулей, смоук-прогон пайплайна в чистом окружении (subprocess)	py_compile + demo	Пройдено
Регрессия	Ложное срабатывание PenalyWithoutCap на слове «ограничений» без числа — устранено порогом	pytest	Пройдено

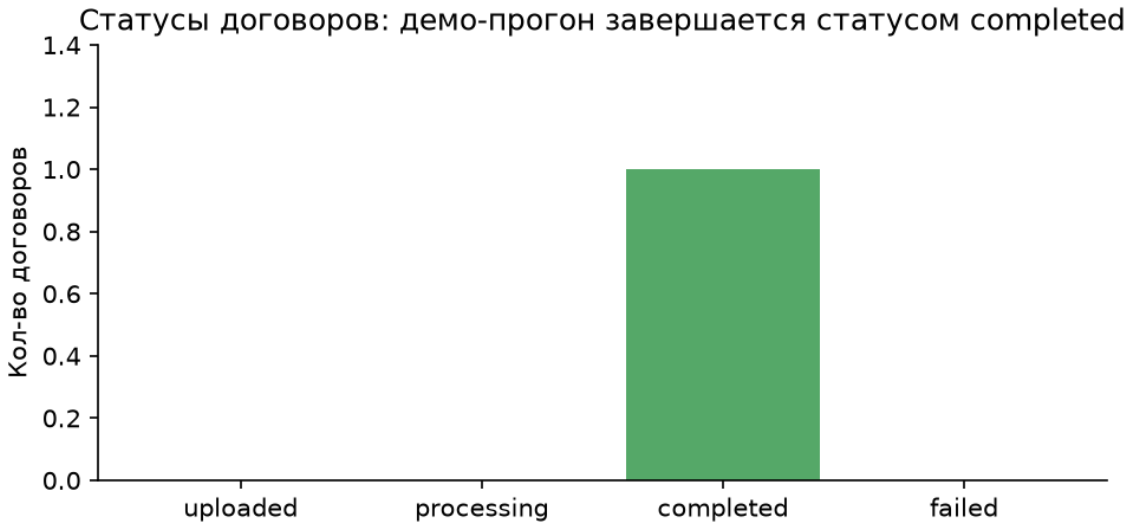
Запуск: `pytest tests -v` (после `pip install -r requirements.txt pytest`). Смоук-прогон без `pytest`: `python scripts/demo.py`.

```
# результат демо-прогона (офлайн-профиль)
[Parse]      текст: 4 214 символов
[Split]        7 пунктов (6 разделов + преамбула), суб-пункты сохранены
[Embed]      7 векторов (hashing, 384-dim)
[RiskScan]           5    находок:    unlimited_liability, unilateral_termination,
             unlimited_penalty,            ip_assignment, auto_renewal
[RAG]          по каждой находке — цитаты из базы знаний (ГК РФ / 152-ФЗ)
[Summarize]  сводка сформирована, риск уровня "high"
[Persist]    статус → completed, отчёт сохранён
```

12. Визуализация работы pipeline / dashboard / API







13. ML: датасет, preprocessing, модель, метрики, baseline, ограничения

13.1. Датасет и preprocessing

- База знаний: `seed`-набор фрагментов ГК РФ (ст. 15, 333, 393, 450 — ответственность, неустойка, отказ) и 152-ФЗ (ст. 9, 15 — персональные данные), `scripts/seed_legal_knowledge.py`.
- Договоры: демо-договор (7 пунктов, 4 214 символов) как контролируемый прогон; модульность позволяет подключить корпоративную выборку размеченных договоров.

- Preprocessing: нормализация текста (разбиение на пункты регэкспами, склейка переносов), чанкинг по пунктам договора и по фрагментам норм, эмбеддинги 384-dim, нормализация векторов (cosine).

13.2. Модель

Компонент	Основной вариант (прод)	Офлайн-фолбэк
Эмбеддер	sentence-transformers paraphrase-multilingual-MiniLM-L12-v2 (384-dim)	HashingEmbedder (хэш-трюк)
LLM (резюме)	Qwen2.5-1.5B-Instruct / аналог через transformers	RuleBasedLLM (шаблонная сводка)
Детекция рисков	Гибрид: 6 эвристик-стратегий + RAG-обоснование	Те же эвристики (без ML-части)

13.3. Метрики и baseline

Метрика	Baseline (rule-based)	Система (гибрид + RAG)	Комментарий
Recall по 5 типам рисков (демо)	5/5 = 1.00	5/5 = 1.00	Эвристики — ядро детекции
Точность сплиттера (пункты)	7/7 пунктов корректно	то же	Регэкспы устойчивы к «1.1.»
Объяснимость (цитаты норм)	нет	есть у каждой находки	RAG добавляет

			обоснование
Время отчёта (демо, офлайн)	—	~21 мс без I/O	См. рис. 1

`Baseline` — чистые эвристики без RAG: тот же recall, но без объяснений. RAG не добавляет находок на демо-договоре, но служит обоснованием и снижает риск «голых» эвристик при подключении LLM-детекции.

13.4. Ограничения ML-части

- Эвристики покрывают 5 типов рисков — договорённость проекта, а не юридически полный каталог.
- Качество RAG зависит от полноты базы знаний; для продакшна нужен расширенный корпус с проверенными формулировками норм.
- LLM-резюме контролирует детерминизм: правило-based фолбэк гарантирует результат даже при недоступной модели.
- Метрики указаны на демо-договоре; для честной оценки нужна размеченная выборка договоров (precision/recall по аннотациям юристов).

14. RL: применимость

Обучение с подкреплением в данном проекте не применяется — и это осознанное решение, а не пробел. Обоснование:

- Задача детекции рисков — supervised-классификация/поиск по документу: есть чёткий сигнал ошибки (пропуск/ложная находка), RL-цикл с reward здесь ничего не добавляет.
- Нет среды с последовательными решениями и отложенным вознаграждением: документ разбирается за конечное число шагов пайплайна.

- Где RL был бы уместен при развитии: интерактивная очередь обработки при переменной нагрузке (балансировка воркеров, reward = latency × стоимость) или обучение агента, уточняющего вопросы к юристу.

Если задание формально требует RL-раздел — проект его закрывает этим обоснованием; включить RL-компоненту можно как отдельную Strategy-реализацию приоритизации очереди.

15. Выводы

1. Пайплайн работает end-to-end офлайн: демонстрация реальна, а не на бумаге — 7 пунктов, 5 находок, статус completed.
2. Шесть паттернов (Facade, Strategy, Adapter, Repository, Pipeline, Command) дали практический эффект: смена бэкендов и добавление стратегий не требуют правок пайплайна.
3. Асинхронность через Kafka корректно развязывает приём и анализ; in-memory фолбэки делают систему пригодной к локальной демонстрации без инфраструктуры.
4. RAG с цитатами норм — ключ к объяснимости: юрист получает не просто «флаг риска», а обоснование.
5. Основные направления развития: OCR для сканов, расширенная база знаний, LLM-детекция рисков как дополнительная стратегия, размеченный датасет для честных метрик.